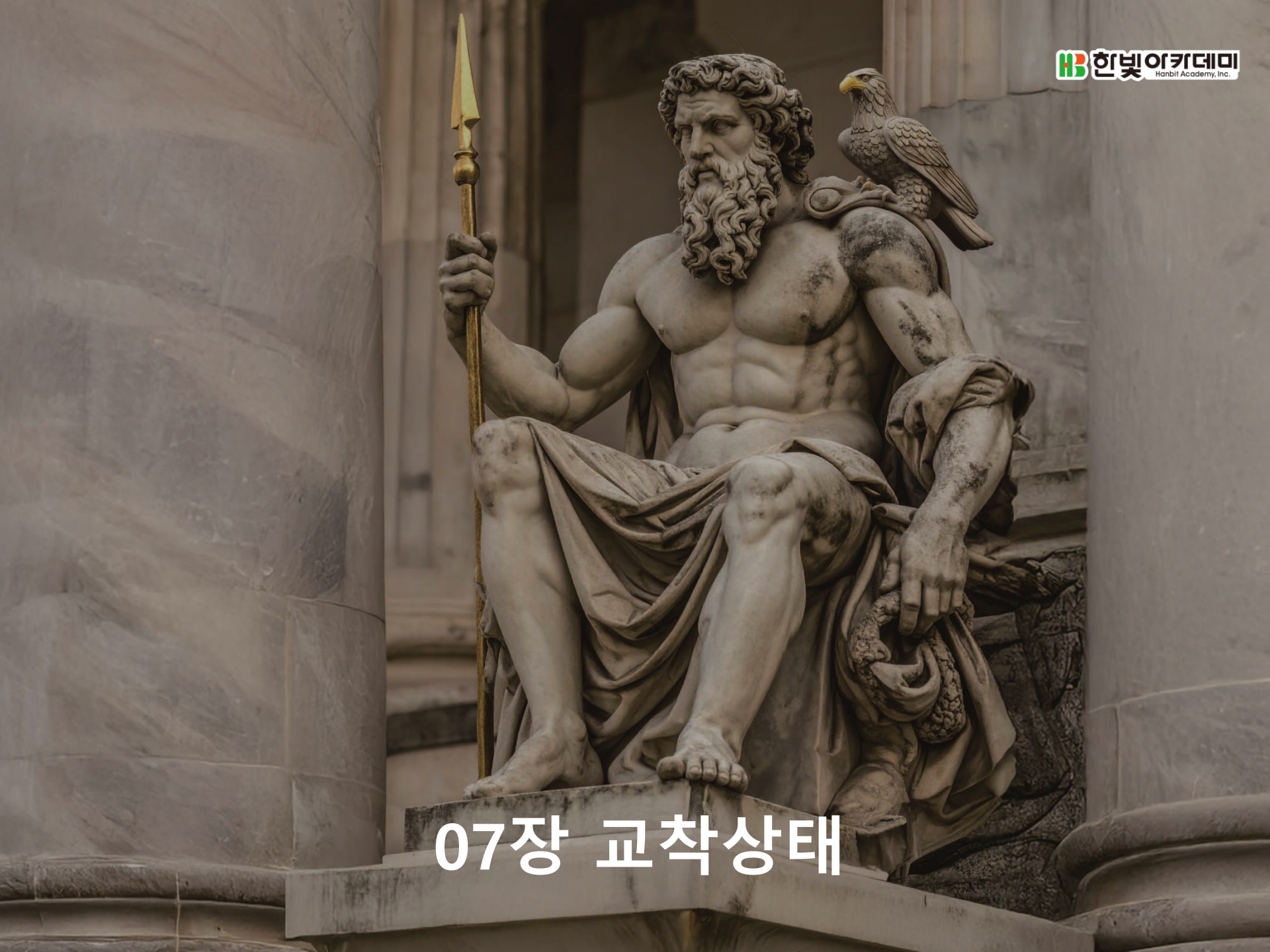


A detailed marble statue of Neptune (or Poseidon) seated on a stone ledge. He has a long, curly beard and hair, and is holding a golden trident in his right hand. A griffon is perched on his left shoulder, and a snake is coiled around his left leg. He is wearing a draped garment around his waist. The background shows classical architectural columns.

07장 교착상태

학습목표

- 교착상태의 개념과 필요조건을 살펴보고
자원할당 그래프로 이를 설명할 수 있다.
- 예방, 회피, 검출, 회복 등 교착상태 처리 전략의 원리와 차이를 이해한다

목차

1. 교착상태의 개념과 필요조건
2. 교착상태 처리 방법

01

교착상태의 개념과 필요조건

01 교착상태의 개념과 필요조건

1. 교착상태의 개념과 사례

◆ 교착상태 개념

- 둘 이상의 프로세스가 각기 자원을 점유한 채, 다른 자원을 기다리면서 작업을 더 이상 진행하지 못하는 상태.



그림 7-1 교통 체증과 교착상태

01 교착상태의 개념과 필요조건

1. 교착상태의 개념과 사례

- 교착상태는 잠금(lock)을 사용하는 모든 시스템에서 발생.

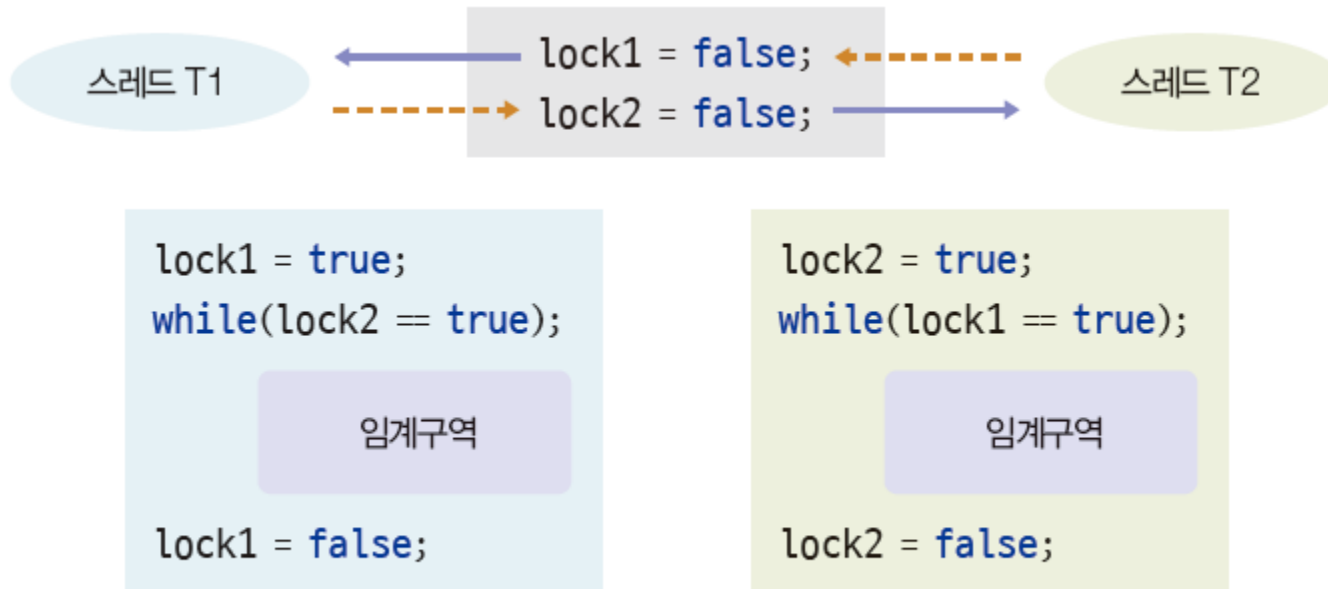
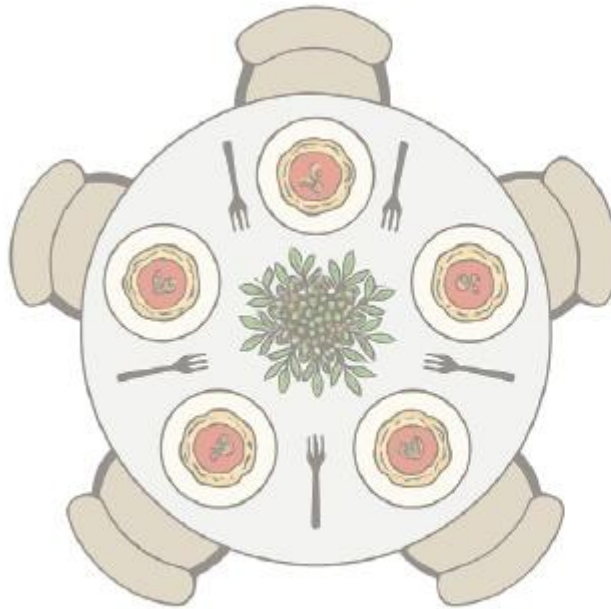


그림 7-2 교착상태를 발생시키는 임계구역 코드

01 교착상태의 개념과 필요조건

1. 교착상태의 개념과 사례

◆ 교착상태 사례: 식사하는 철학자 문제



모든 철학자가 동시에 왼쪽
포크를 잡으면 아무도 오른쪽
포크를 잡을 수 없다.



그림 7-3 식사하는 철학자 문제

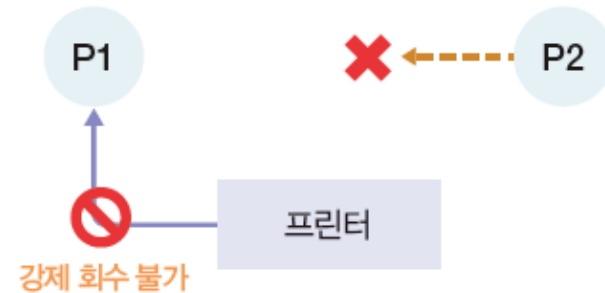
01 교착상태의 개념과 필요조건

2. 교착상태 필요조건

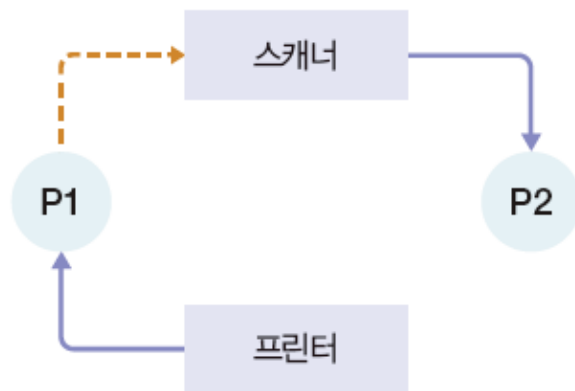
◆ 교착상태의 4가지 필요조건



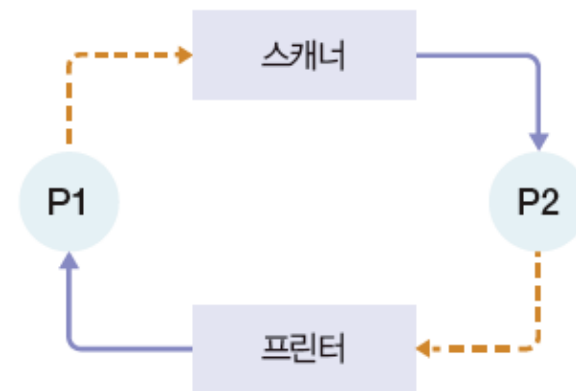
(a) 상호 배제



(b) 비선점



(c) 점유와 대기



(d) 순환 대기

그림 7-4 교착상태의 4가지 필요조건

01 교착상태의 개념과 필요조건

2. 교착상태 필요조건

◆ 교착상태의 4가지 필요조건

- 상호 배제: 자원은 한 번에 하나의 프로세스만 사용할 수 있어야 함.
- 비선점: 한 프로세스가 점유한 자원을 강제로 빼앗을 수 없음.
- 점유와 대기: 자원을 보유하고 있는 프로세스는 다른 자원을 기다리는 상태여야 함.
- 순환 대기: 프로세스 간에 자원을 기다리는 형태가 순환 구조를 이루어야 함.

01 교착상태의 개념과 필요조건

2. 교착상태 필요조건

◆ 교착상태 분석을 위한 자원할당 그래프

- 자원할당 그래프: 프로세스가 어떤 자원을 사용하고 있으며, 어떤 자원을 기다리고 있는지를 표현한 방향 그래프

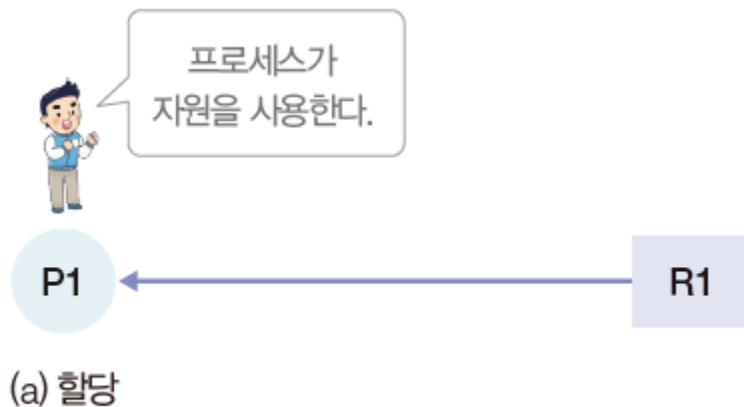


그림 7-5 자원할당 그래프

01 교착상태의 개념과 필요조건

2. 교착상태 필요조건

◆ 교착상태 분석을 위한 자원할당 그래프

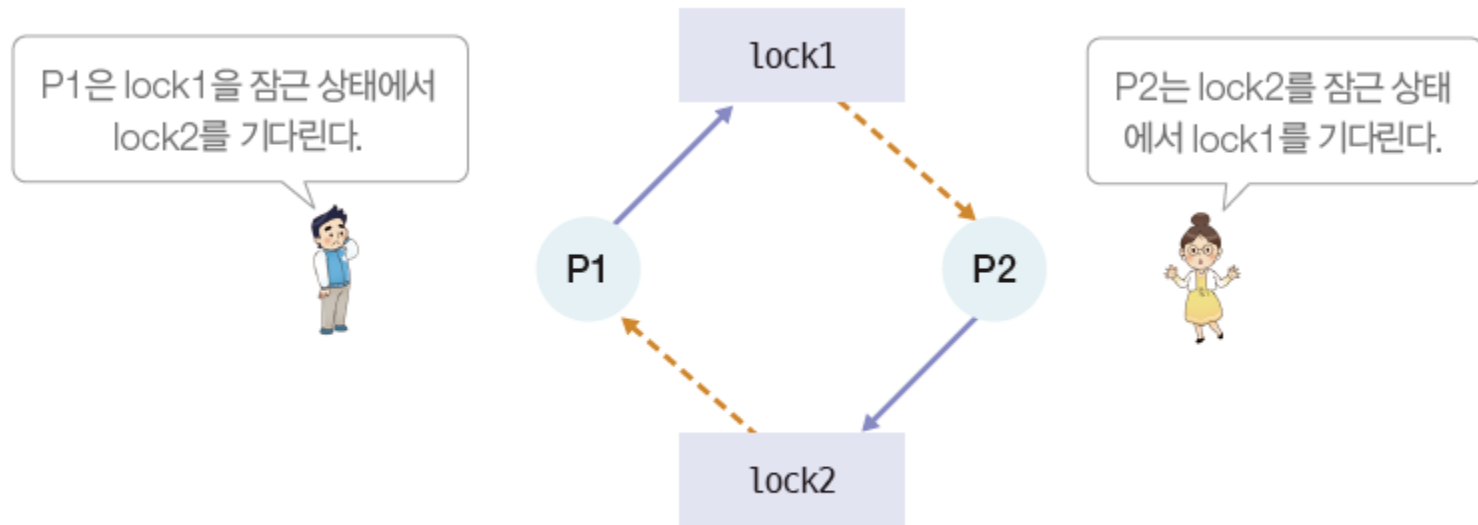


그림 7-6 한정 대기 조건을 위반하는 임계구역 문제의 자원할당 그래프

01 교착상태의 개념과 필요조건

2. 교착상태 필요조건

◆ 교착상태 분석을 위한 자원할당 그래프

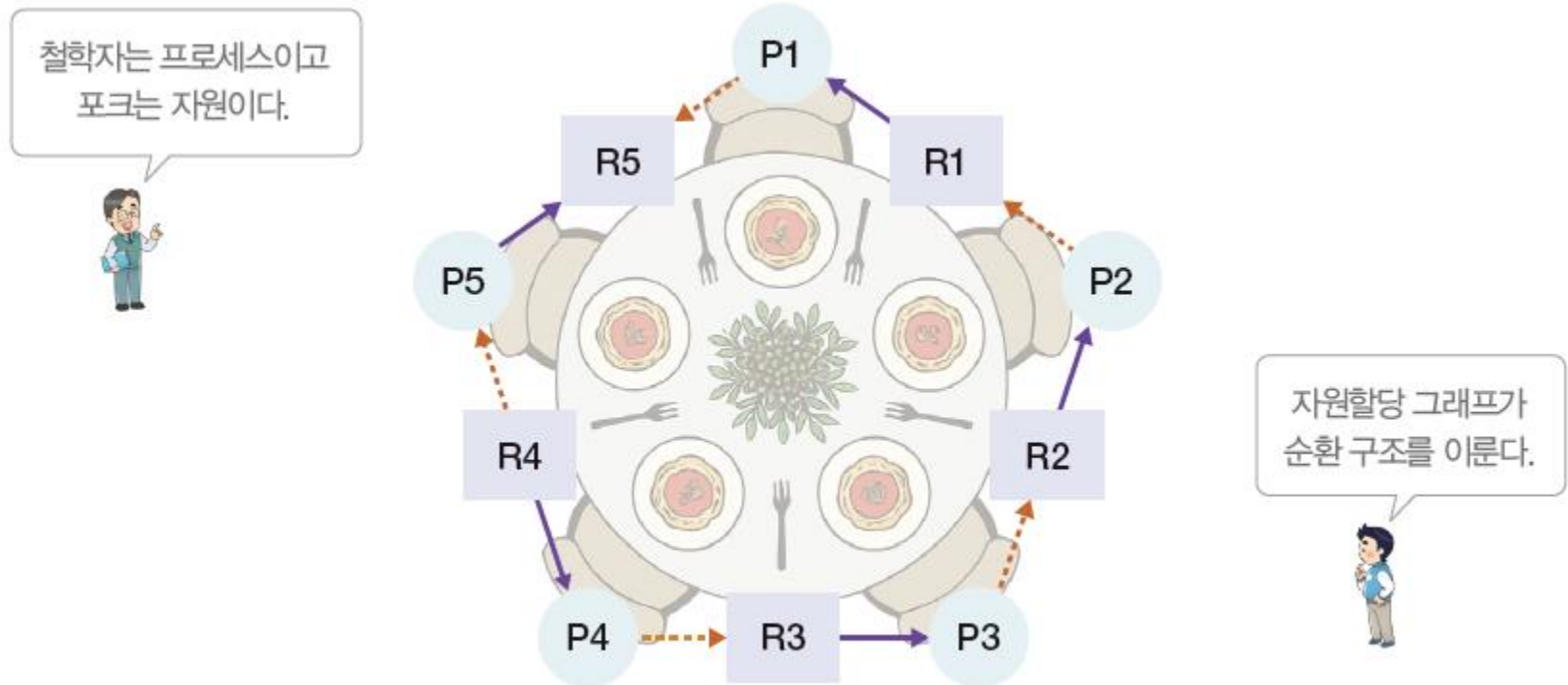


그림 7-7 식사하는 철학자 문제의 자원할당 그래프

02

교착상태 처리 방법

02 교착상태 처리 방법

1. 교착상태 처리 방법의 종류

- 교착상태 예방
- 교착상태 회피
- 교착상태 검출과 회복

02 교착상태 처리 방법

2. 교착상태 예방

◆ 상호 배제 예방

- 상호 배제 조건을 사전에 제거하여 교착상태 발생을 막는 방식.
- 시스템 내의 자원을 여러 프로세스가 동시에 사용할 수 있도록 공유 자원으로 설계.
- 그러나 현실적으로 모든 자원을 공유하는 것은 불가능.

02 교착상태 처리 방법

2. 교착상태 예방

◆ 비선점 예방

- 비선점 조건을 사전에 제거하여 교착상태 발생을 막는 방식.
- 프로세스가 사용중인 자원을 운영체제가 강제로 회수할 수 있도록 하여 교착상태를 방지.
- 기아 현상이 발생 가능성 존재.

02 교착상태 처리 방법

2. 교착상태 예방

◆점유와 대기 예방

- 점유와 대기 조건을 사전에 제거하여 교착상태 발생을 막는 방식.
- 프로세스가 자원을 점유한 상태에서는 추가 자원을 요청하지 못함.

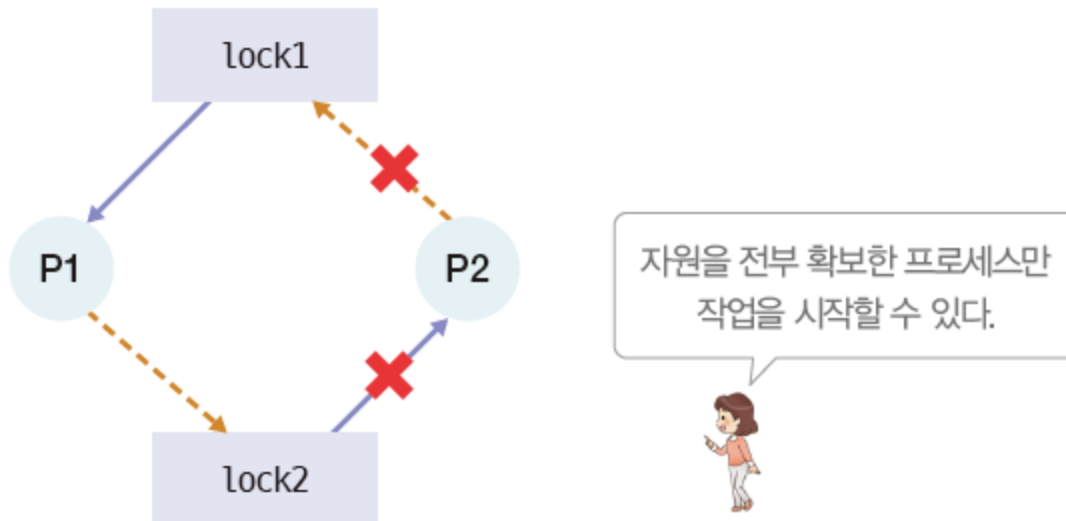


그림 7-8 점유와 대기 예방

02 교착상태 처리 방법

2. 교착상태 예방

◆점유와 대기 예방

- 프로세스 실행 전에 필요한 모든 자원을 미리 알기 어려움.
- 자원을 미리 점유하면 자원 활용률이 낮아짐.
- 많은 자원을 사용하는 프로세스는 실행을 시작하기가 더욱 어려움.
- 결국 일괄 작업 방식이 되어 시스템 효율이 떨어짐.

02 교착상태 처리 방법

2. 교착상태 예방

◆순환 대기 예방

- 순환 대기 조건을 사전에 제거하여 교착상태 발생을 막는 방식.
- 자원에 번호를 부여 후, 번호가 증가하는 방향으로만 자원을 요청할 수 있도록 제한.

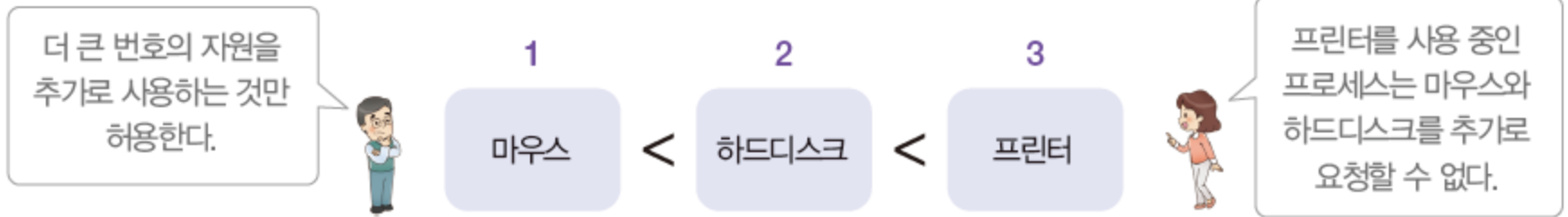


그림 7-9 순환 대기 예방

02 교착상태 처리 방법

2. 교착상태 예방

◆순환 대기 예방

- 프로세스 작업의 유연성이 떨어짐
- 자원의 번호를 어떻게 부여할 것인지 결정하기 어려움

표 7-1 교착상태 예방 방법

교착상태 예방 방법	설명	문제점
상호 배제 예방	모든 자원을 여러 프로세스가 동시에 사용할 수 있도록 설계	경쟁 상태 발생
비선점 예방	자원을 사용 중인 프로세스에서 운영체제가 자원을 회수할 수 있도록 함	기아 현상 발생
점유와 대기 예방	필요한 모든 자원을 한꺼번에 확보하지 못하면 실행하지 않음	자원 효율성 저하
순환 대기 예방	자원에 번호를 부여하고 번호가 증가하는 방향으로만 요청하게 함	작업 진행의 유연성 저하

02 교착상태 처리 방법

3. 교착상태 회피

◆ 교착상태 회피의 개념

- 프로세스에 자원을 할당할 때, 교착상태가 발생할 가능성을 먼저 평가하고 안전한 범위 내에서만 자원을 할당하는 방법.
- 교착상태가 발생할 가능성이 있다면 자원 할당을 중지.

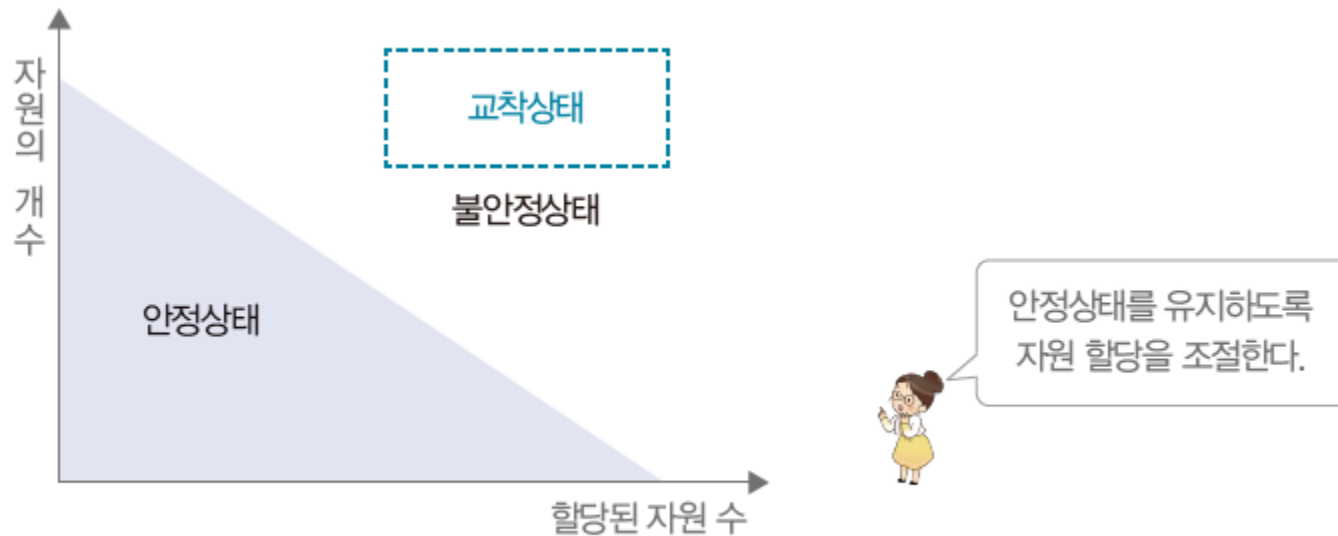


그림 7-10 안정상태와 불안정상태

02 교착상태 처리 방법

3. 교착상태 회피

◆은행원 알고리즘

- 교착상태 회피를 구현하는 대표적인 방법.
- 교착상태가 발생하지 않을 것이라고 판단하면 자원을 할당.

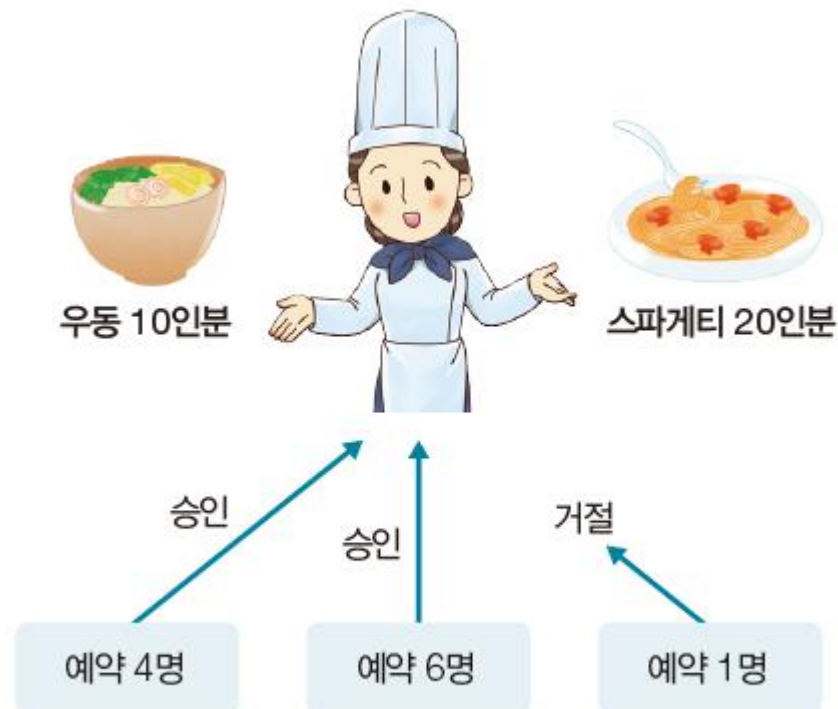


그림 7-11 음식점의 은행원 알고리즘

02 교착상태 처리 방법

3. 교착상태 회피

◆은행원 알고리즘

표 7-2 은행원 알고리즘의 변수

변수	설명
전체 자원	시스템 내 전체 자원의 수
가용 자원	시스템 내 현재 사용할 수 있는 자원의 수 (가용 자원 = 전체 자원 - 모든 프로세스의 할당 자원)
최대 자원	각 프로세스가 선언한 최대 자원의 수
할당 자원	각 프로세스에 현재 할당된 자원의 수
기대 자원	각 프로세스가 앞으로 사용할 자원의 수 (기대 자원 = 최대 자원 - 할당 자원)

02 교착상태 처리 방법

3. 교착상태 회피

◆은행원 알고리즘

- 안정상태: 각 프로세스의 기대 자원과 비교하여 가용 자원이 크거나 같은 경우.

안정상태에서는 가용 자원을 할당하여 프로세스를 1개라도 종료할 수 있다.



Total = 14 Available = 2			
Process	Max	Allocation	Expect
P1	5	2	3
P2	6	4	2
P3	10	6	4

P2에 가용 자원을 할당하면 안정상태를 유지할 수 있다.



그림 7-12 안정상태

02 교착상태 처리 방법

3. 교착상태 회피

◆은행원 알고리즘

- 불안정상태: 기대 자원을 충족하지 못하여 작업을 마칠 수 없는 상태.

Total = 14 Available = 1			
Process	Max	Allocation	Expect
P1	7	3	4
P2	6	4	2
P3	10	6	4

가용 자원으로 어떤 프로세스도 끝낼 수 없으므로 자원을 할당하지 않는다.



그림 7-13 불안정상태

02 교착상태 처리 방법

3. 교착상태 회피

◆ 회피 기법의 한계

- 프로세스가 사용할 모든 자원을 미리 선언해야 함.
- 시스템의 전체 자원 수가 고정적이어야 함.
- 자원을 낭비할 가능성이 높음.

02 교착상태 처리 방법

4. 교착상태 검출

◆ 교착상태 무시

- 교착상태를 명시적으로 처리하지 않고 무시하는 것.
- 교착상태 발생 빈도가 낮고, 영향이 크지 않은 일반적인 환경에서 사용.

◆ 타임아웃 기반 간접 검출

- 일정 시간 동안 응답하지 않는 프로세스를 교착상태에 빠졌다고 간주, 해당 프로세스를 종료하거나 사용자에게 종료 여부를 선택하게 하는 방식.

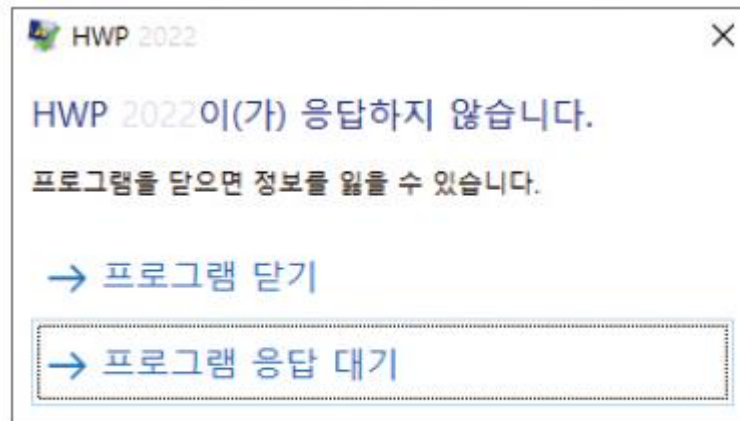


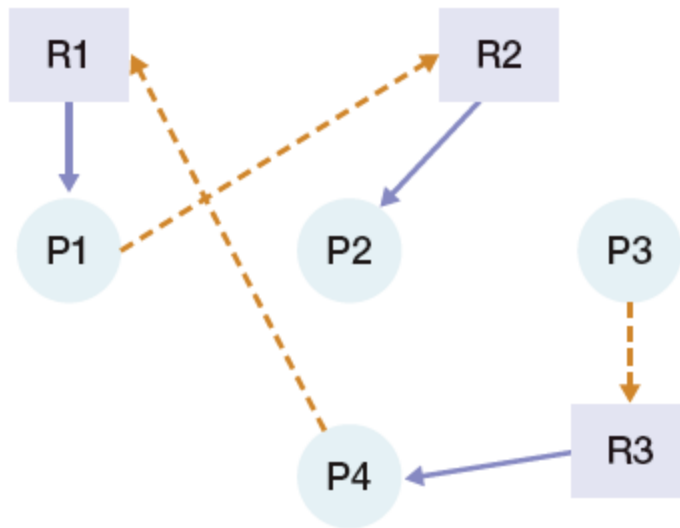
그림 7-14 타임아웃 프로세스 종료 알림

02 교착상태 처리 방법

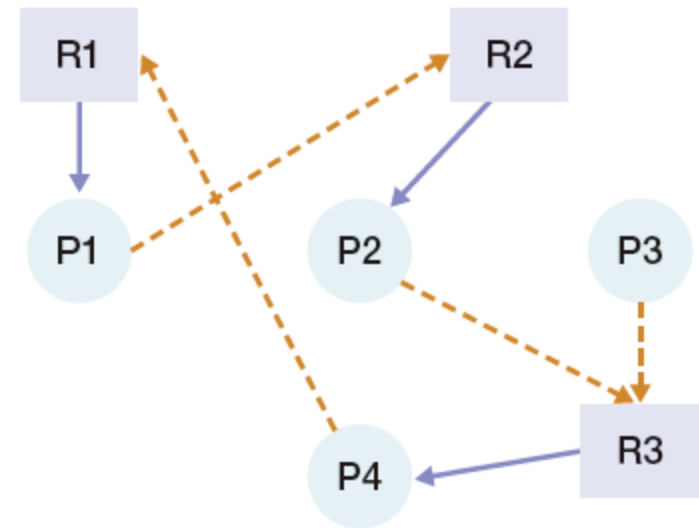
4. 교착상태 검출

◆ 자원할당 그래프 기반 검출

- 프로세스와 자원 사이의 요청 및 할당 관계를 방향 그래프로 나타냄.



(a) 순환 구조 없음—교착상태 아님



(b) 순환 구조 있음—교착상태

그림 7-15 자원할당 그래프와 교착상태

02 교착상태 처리 방법

5. 교착상태 회복

◆ 프로세스 강제 종료

- 교착상태 회복: 검출된 교착상태를 해결하기 위해 프로세스를 강제로 종료.
- 프로세스의 종료 기준
 - 우선순위를 고려
 - 우선순위가 같을 때 작업 시간을 고려
 - 우선순위와 작업 시간이 같을 때 자원 사용량 고려

02 교착상태 처리 방법

5. 교착상태 회복

◆ 프로세스 강제 종료

- 체크포인트: 문제가 발생했을 때 안전한 상태로 복구할 수 있도록 특정 시점의 데이터를 저장하는 작업.
- 롤백: 문제가 발생한 후 체크포인트로 설정된 시점으로 시스템을 되돌리는 작업

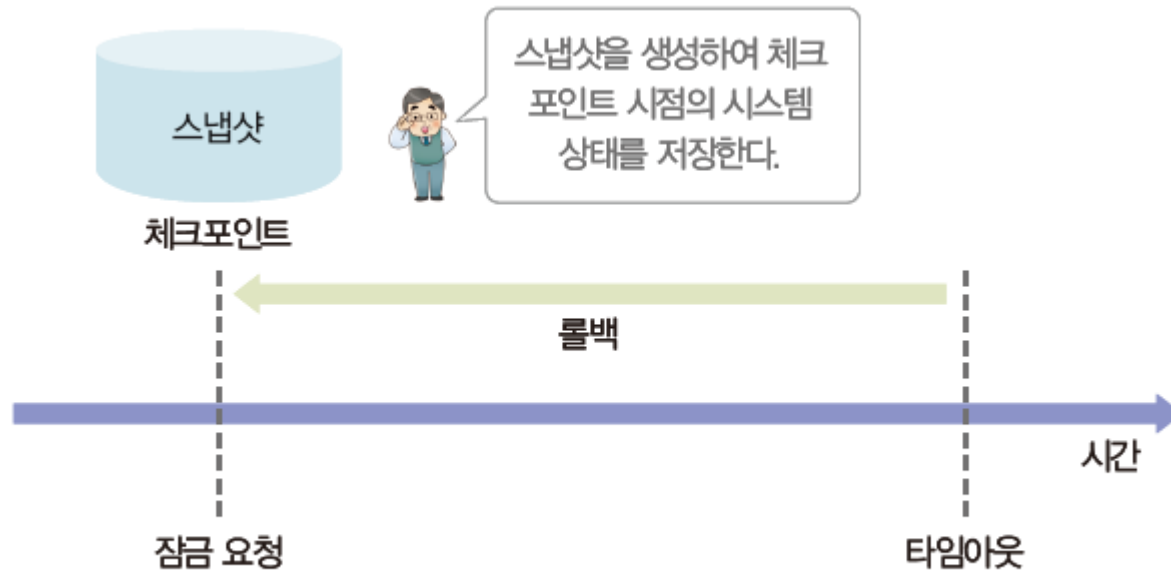


그림 7-16 시스템 복원

02 교착상태 처리 방법

5. 교착상태 회복

◆ 프로세스 강제 종료

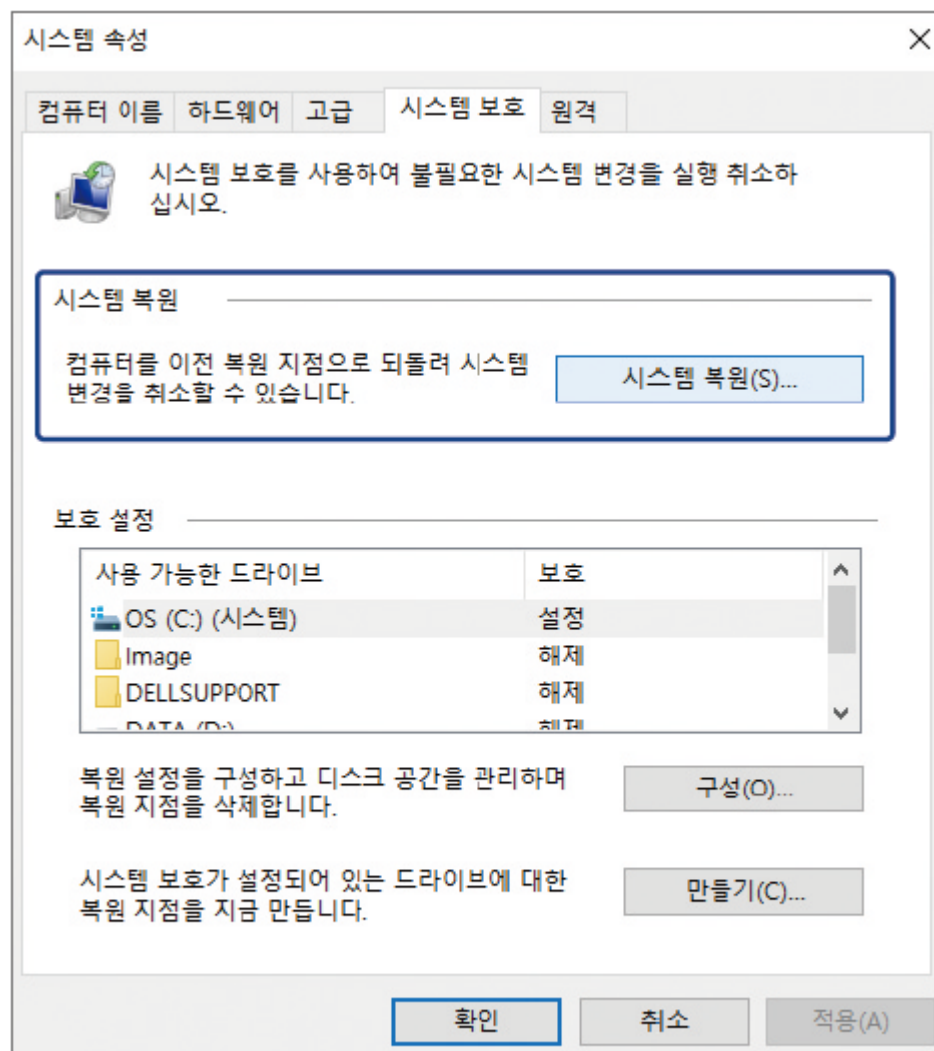


그림 7-17 윈도우의 시스템 복원 기능

Thank You!